

The Art of the Fugue: Minimizing Interleaving in Collaborative Text Editing

Matthew Weidner^{ID} and Martin Kleppmann

Abstract—Most existing algorithms for replicated lists, which are widely used in collaborative text editors, suffer from a problem: when two users concurrently insert text at the same position in the document, the merged outcome may interleave the inserted text passages, resulting in corrupted and potentially unreadable text. The problem has gone unnoticed for decades, and it affects both CRDTs and Operational Transformation. This paper defines maximal non-interleaving, our new correctness property for replicated lists. We introduce two related CRDT algorithms, Fugue and FugueMax, and prove that FugueMax satisfies maximal non-interleaving. We also implement our algorithms and demonstrate that Fugue offers performance comparable to state-of-the-art CRDT libraries for text editing.

Index Terms—Distributed data structures, replica consistency, collaborative text editing, Conflict-free replicated data types (CRDTs), operational transformation.

I. INTRODUCTION

COLLABORATIVE text editors such as Google Docs allow several users to concurrently modify a document, while ensuring that all users’ copies of the document converge to the same state. Even though algorithms for collaborative text editing have been studied for over three decades [1], [2], [3], a formal specification of the required behavior only appeared as recently as 2016 [4]. We argue that this specification is incomplete. There is an additional correctness property that is important in practice, but which has been overlooked by almost all prior research on this topic: *non-interleaving*. Informally stated, this property requires that when sections of text are composed independently from each other (perhaps while the users are offline), and the edits are subsequently merged, those sections are placed one after another, and not intermingled in the final document.

For example, suppose two users are editing a text document containing a shopping list, as shown in Fig. 1. Initially, the document contains the word “milk” and a newline character. User A inserts a line break and “eggs”, while concurrently (due to a weak network connection), user B inserts a line break and

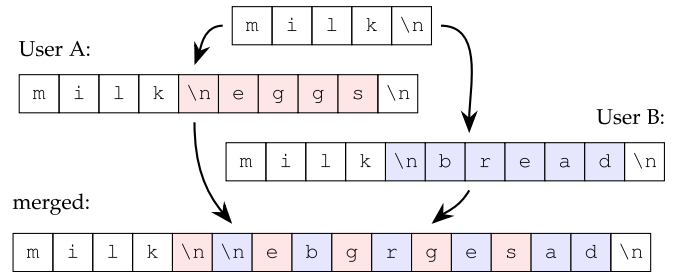


Fig. 1. Possible interleaving in a collaborative text editor.

“bread”. A collaborative text editor that is correct with respect to the existing formal specification may choose to merge their edits as shown: “milk”, a blank line, and then the interleaved word “ebgrgesad”.

Several existing text collaboration algorithms interleave text in this way, and almost all algorithms that we surveyed interleave text in some cases. Affected algorithms include both main approaches to collaborative text editing: Conflict-free Replicated Data Types (CRDTs) [5], [6], and Operational Transformation (OT) [1], [3].

This paper describes the interleaving problem in detail and introduces novel algorithms that minimize interleaving. One surprising result of this paper is that it is impossible to avoid interleaving in every situation. Users can perform multiple interacting concurrent updates in such a way that *any* algorithm must interleave some characters in the merged text, at least according to a naive definition that forbids interleaving both forward and backward insertions (defined in Section II-C). Thus we instead define a new correctness property, *maximal non-interleaving*, which forbids interleaving to a maximum possible extent.

We then introduce *Fugue* and *FugueMax*, two novel CRDT algorithms for collaborative text editing. We prove that both algorithms avoid interleaving text except in the rare situations where some interleaving is inevitable. Specifically, *FugueMax* satisfies maximal non-interleaving, while *Fugue* is simpler but may interleave more characters than necessary in situations where interleaving is inevitable. Finally, we provide an optimized open source implementation of *Fugue* and show that it achieves performance comparable to the state-of-the-art Yjs library on a realistic text-editing trace.

II. BACKGROUND

CRDT and OT algorithms for text allow multiple users, each with their own copy of the document, to concurrently edit the text. Although these algorithms are usually presented in the

Received 20 November 2023; revised 12 May 2025; accepted 15 September 2025. Date of publication 26 September 2025; date of current version 2 October 2025. The work of Matthew Weidner was supported by NDSEG Fellowship sponsored by the US Office of Naval Research. The work of Martin Kleppmann was supported by Volkswagen Foundation and Crowdfunding Supporters including SoftwareMill. Recommended for acceptance by O. Ozkasap. (*Corresponding author: Matthew Weidner.*)

Matthew Weidner is with Carnegie Mellon University, Pittsburgh, PA 15213 USA (e-mail: maweidne@andrew.cmu.edu).

Martin Kleppmann is with the University of Cambridge, CB2 1TN Cambridge, U.K.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPDS.2025.3611880>, provided by the authors.

Digital Object Identifier 10.1109/TPDS.2025.3611880

context of text editing, they can easily be generalized beyond text: instead of a list of characters, the algorithm could manage a list of other objects, such as items on a to-do list, or rows in a spreadsheet. We therefore also say that these algorithms implement a replicated list object [4].

A. System Model

In collaborative text editors, each user session (e.g., a tab in a web browser) maintains a replica of the list of characters. On user input, the user's local replica of the document is updated by inserting or deleting characters in this list, and each such insertion or deletion is called an *operation*. A user's operations are immediately applied to their local replica, without waiting for network communication with any other nodes, in order to provide responsive user interaction independently of network latency. This model also allows disconnected operation: if a user edits the document while offline, their client buffers the operations they generate and sends them to collaborators when they next connect.

Assuming users eventually come online, every operation is eventually propagated to other replicas, and each replica integrates remote operations into its local state as they are received. At a minimum, this process must ensure *convergence*: any two replicas that have processed the same set of operations must be in the same state, even if they received the operations in a different order. A more detailed specification of a replicated list is given by Attiya et al. [4]; we summarize it in the proof of Theorem 1 in Section IV.

Algorithms for collaborative text editing differ in the assumptions they make about the network between replicas. For example, the OT algorithm Jupiter [2] assumes that all operations pass through a central server that sequences and transforms those operations. On the other hand, CRDTs (including our Fugue algorithm in Section IV) generally assume a causal broadcast protocol [5]. Causal broadcast can be implemented in a peer-to-peer network without assuming any central server or consensus protocol [7], [8], making this model suitable for decentralized systems. The causal broadcast protocol handles retransmission of dropped messages and ensures that when a replica comes online, it receives all the operations it missed while offline.

B. The Interleaving Problem

Several replicated list CRDTs, including Treedoc [9], Loggoot [10], and LSEQ [11], assign to each list element a unique identifier from a dense, totally ordered set. The sequence of list elements is then obtained by sorting the IDs in ascending order. To insert a new list element between two adjacent elements with IDs id_1 and id_2 respectively, the algorithm generates a new unique ID id_3 such that $id_1 < id_3 < id_2$, where $<$ is the total order on identifiers.

Say another user concurrently inserts an element with ID id_4 between the same pair of elements (id_1, id_2) such that $id_1 < id_4 < id_2$. The minimum requirement of $id_3 \neq id_4$ is easy to achieve (e.g., by including in each ID the unique name of the replica that generated it), but whether $id_3 < id_4$ or $id_3 > id_4$ is an arbitrary choice.

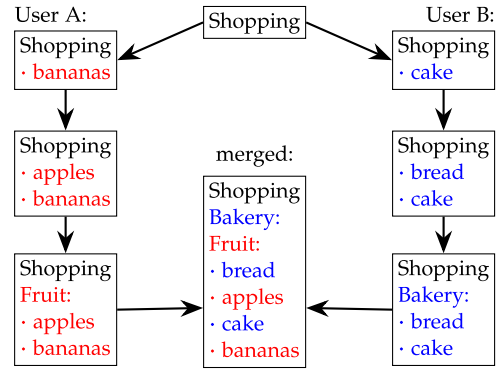


Fig. 2. Each user prepends items to their shopping list. When the edits are merged in an algorithm that allows interleaving of backward insertions, the result may place the items in an illogical order, such as listing bread in the fruit category.

When two users concurrently insert several new elements in the same ID interval, the result is the effect illustrated in the introduction's Fig. 1. For example, in the Treedoc list CRDT, each ID is a path through a binary tree together with a tiebreaker. When two users insert at the same location, they choose the same paths through the binary tree: 'e' in "eggs" and 'b' in "bread" use the same path, as do 'g' in "eggs" and 'r' in "bread", etc. The algorithm then visits each path in tree traversal order; with tiebreakers, the merged result thus alternates between the letters of each word: "ebgrgesad".

We argue that this behavior is obviously undesirable. Nevertheless, none of the affected papers even mention the issue, and although it had been informally known to people in the field for some time, it was not documented in the research literature until 2018 [12], [13]. Attiya et al.'s specification [4] allows interleaving like in Fig. 1.

C. Interleaving of Forward and Backward Insertions

In some replicated list algorithms, whether interleaving can occur or not depends on the order in which the elements are inserted into the list. In the common case, when a user writes text, they insert characters in forward direction: that is, "bread" is inserted as the character sequence "b", "r", "e", "a", "d". However, not all writing is in forward direction: sometimes users hit backspace to fix a typo, or move their cursor to a different location in the document and continue typing there.

Besides inserting characters in forward direction, it is also possible to insert in backward direction. The extreme case of typing text in reverse order character by character (typing "bread" as "d", "a", "e", "r", "b") is unlikely to occur in practical text editing scenarios. However, a plausible scenario of backward insertion is illustrated in Fig. 2. In this example, two users add items to a shared shopping list while offline. Each user prepends new items to the beginning of the list, finally adding a category header ("Fruit" or "Bakery") for the items they added. When the users merge their changes, an algorithm that allows interleaving of backward insertions may place the items in a surprising order. This behavior is less bad than the

fine-grained character-by-character interleaving of Fig. 1, but it is nevertheless not ideal. It would be preferable to keep all of each user’s insertions as one contiguous string, regardless of the order in which the elements were inserted.

When OT/CRDT algorithms for replicated lists are used for data other than text, backward insertion is more likely to occur. For example, in a spreadsheet or to-do list, new rows/items might regularly be inserted at the top, one at a time. If we can avoid both forward and backward interleaving, we also improve the behavior of these applications.

III. RELATED WORK

Collaborative text editing originated with the work of Ellis and Gibbs [1], who also introduced Operational Transformation (OT) as a technique for resolving concurrent edits. This approach was formalized by Ressel et al. [14], and further developed by Sun et al. [15] and many other papers. The OT algorithm Jupiter [2] later became the basis for real-time collaboration in Google Docs [16]. Following bugs in several OT algorithms, which failed to converge in some situations [17], [18], Conflict-free Replicated Data Types (CRDTs) were developed as an alternative approach [5]. The first CRDT for text editing was WOOT [19], which was followed by Treedoc [9], Logoot [10], RGA [20], and others.

A. Algorithms That Exhibit Interleaving

The interleaving problem was first noticed in CRDTs such as Logoot and LSEQ because they are particularly prone to the problem; experiments with implementations of these algorithms are easily able to trigger interleaving in practice [21]. However, when we started looking at the issue more closely, we found that interleaving is surprisingly prevalent among both OT and CRDT algorithms for collaborative text editing. Our findings are summarized in Table I, and examples of each instance of interleaving are detailed in the appendix.

Occurrence of interleaving is often nondeterministic, and the probability of exhibiting interleaving varies depending on the algorithm: for example, in some algorithms it depends on the exact order in which concurrently sent network messages are received, and in some it depends on random numbers generated as part of the algorithm.

In some algorithms, interleaving occurs only if multiple replicas participate in one of the concurrent editing sessions; this is indicated in the column labeled “multi-replica” in Table I. This can happen, for example, if a user starts some work on one device and then continues on another device (producing an editing session that spans two devices), while independently another user works offline on the same document on a third device. It can also occur in systems with ephemeral replica IDs, such as a web application that generates a fresh ID for every browser tab refresh.

In the cases marked $\circ$ in Table I we conjecture non-interleaving, but we have not proved it. Only in the cases marked $\circ\checkmark$ has non-interleaving been proven. RGA forward non-interleaving was proved by Kleppmann et al. [12], Yjs forward non-interleaving is proved in an in-progress paper,

TABLE I
VARIOUS ALGORITHMS’ SUSCEPTIBILITY TO INTERLEAVING ANOMALIES. KEY:
● = INTERLEAVING CAN OCCUR; ○ = WE HAVE NOT BEEN ABLE TO FIND
EXAMPLES OF INTERLEAVING; $\circ\checkmark$ = PROVEN NOT TO INTERLEAVE; $\frac{1}{2}$ =
ALGORITHM MAY INCORRECTLY REORDER CHARACTERS. EXAMPLES OF
ANOMALIES APPEAR IN THE APPENDIX

Family	Algorithm	forward interleaving (one replica)	backward interleaving (one replica)	backward interleaving (multi- replica)
OT	adOPTed [14]	●	○	●
	Jupiter [2]	●	○	○
	GOT [3]	●	$\frac{1}{2}$	$\frac{1}{2}$
	SOCT2 [22]	●	●	●
	TTF [23]	●	○	●
CRDT	WOOT [19]	●	○	○
	Logoot [10]	●	●	●
	LSEQ [11]	●	●	●
	Treedoc [9]	●	●	●
	RGA [20]	$\circ\checkmark$	●	●
	Yjs [24]	$\circ\checkmark$	○	●
	Sync9 [25]	○	○	○
	YjsMod [26]	○	○	○
	Fugue	$\circ\checkmark$	$\circ\checkmark$	$\circ\checkmark$
	FugueMax	$\circ\checkmark$	$\circ\checkmark$	$\circ\checkmark$

and our own algorithms Fugue and FugueMax are verified in Section V. (For the table’s claims of backward non-interleaving, we exclude situations where some interleaving is inevitable for any algorithm; see Section V-B.)

The only existing algorithms for which we have not found interleaving examples are Sync9 [25], and a modification of Yjs proposed by Seph Gentle (“YjsMod”) [26]. At the time of writing, there are no research papers describing these algorithms, and no proofs of correctness; the only published material is source code and informal documentation [25], [26]. Fugue was developed independently from both of these algorithms, while FugueMax uses a technique from YjsMod (see Section V-C). We conjecture that Sync9 is semantically equivalent to Fugue, while YjsMod is semantically equivalent to FugueMax. If that is indeed the case, then Sync9 and YjsMod satisfy forward and backward non-interleaving.

B. Previous Attempt to Ensure Non-Interleaving

Kleppmann et al. [27] previously identified the interleaving problem. That work has two serious flaws:

- 1) The definition of non-interleaving in that paper cannot be satisfied by any algorithm.
- 2) The CRDT algorithm proposed in that paper, which aims to be non-interleaving, is incorrect – it does not converge. An example of non-convergence, which was found by Chandrassery [28], is given in the appendix.

They define non-interleaving as follows (paraphrased):

Suppose two sets of list elements X and Y satisfy:

- All elements in X were inserted concurrently to all elements in Y .

- The elements were inserted at the same location in the document, that is: after applying the insertions for $X \cup Y$ and their causal predecessors, $X \cup Y$ are contiguous in the list order.

Then either X appears before Y or vice-versa. That is, either $\forall x \in X, y \in Y. x \prec y$ or $\forall x \in X, y \in Y. y \prec x$, where $\prec$ is the order of elements in the final list.

To show that no replicated list algorithm can satisfy this definition, it is sufficient to give a counterexample. Starting from an empty list, suppose four replicas concurrently each insert one element. After applying these four insertions, the list state must be some ordering of these four elements; let the order be $abcd$. Then $X = \{a, c\}$ and $Y = \{b, d\}$ satisfy the two hypotheses, but they are interleaved. Since this situation could arise with any algorithm, it cannot be prevented.

In Section V-B we give a new definition of non-interleaving that can be implemented, and we prove that our FugueMax algorithm implements it.

IV. THE FUGUE ALGORITHM

We now introduce *Fugue* (pronounced [fju:g]), a new algorithm for replicated lists and collaborative text editing. It is named after a form of classical music in which several melodic lines are interwoven in a pleasing way. We analyze Fugue's non-interleaving properties in Section V, and we evaluate implementations in Section VI. The FugueMax algorithm is a slight modification of Fugue which we defer until Section V-C. Algorithm 1 gives pseudocode for Fugue.

We describe Fugue as an operation-based CRDT [5], although it can easily be reformulated as a state-based CRDT. The external interface of Fugue is an ordered sequence of values, e.g., the characters in a text document. Since the same value may appear multiple times in a list, we use *element* to refer to a unique instance of a value. Then the operations on the list are:

- $\text{insert}(i, x)$ (Algorithm 1, lines 21–38): Inserts a new element with value x at index i , between existing elements at indices $i - 1$ and i . All later elements (index $\geq i$) shift to an incremented index.
- $\text{delete}(i)$ (Algorithm 1, lines 39–44): Deletes the element at index i . All later elements (index $\geq i + 1$) shift to a decremented index.

Note that we omit operations to mutate or move elements; these can be implemented by combining a replicated list with other CRDTs [29]. We also omit optimizations that compress consecutive runs of insertions or deletions; these can be added later without affecting the core algorithm. At a high level the algorithm works as follows:

State: The state of each replica is a tree in which each non-root node is labeled with a unique ID and a value (Algorithm 1, line 10). Each non-root node is marked as either a *left* or *right* child of its parent, but the tree is not necessarily binary: a parent can have multiple left children or right children, as illustrated in Fig. 3. The tree does not need to be balanced.

Each non-root node in the tree corresponds to an element in the list (e.g., a character in the text document). The list order is given by the depth-first in-order traversal over this tree: first recursively traverse a node's left children, then visit the node's own value, then traverse its right children (Algorithm 1, lines

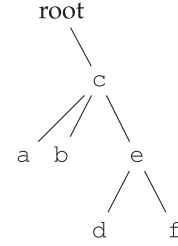


Fig. 3. One possible Fugue structure for the list $abcdef$. Observe that a and b are both left children of c ; they are sorted lexicographically by their elements' IDs.

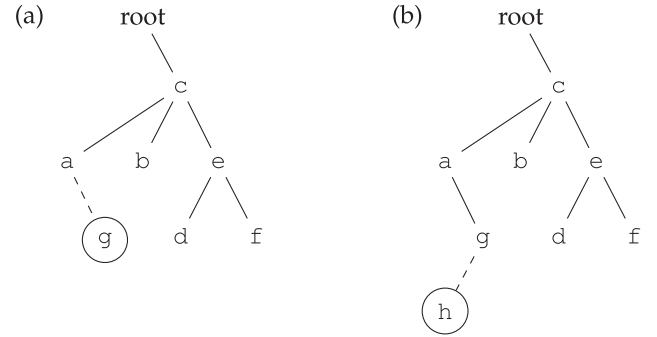


Fig. 4. (a) Inserting a new element g between a and b . When a has no right children, making g a right child of a places it immediately after a in the traversal. (b) Inserting a second new element h between a and g . When g is a descendant of a , we make h a left child of g .

12–20). *Same-side siblings*—nodes with the same parent and the same side—are traversed in lexicographic order of their IDs; the exact construction of IDs and their order is not important.

Insert: To implement $\text{insert}(i, x)$, a replica creates a new node, labeled with a new unique ID and value x , at an appropriate position in its local tree: if the element at index $i - 1$ has no right children, the new node becomes a right child of the element at index $i - 1$ (lines 25–26); otherwise, the new node is added as a left child of the next element in the traversal order (lines 27–28). Fig. 4 illustrates how this choice is made, and Theorem 1 shows that this approach results in the desired behavior. The replica then uses a causal broadcast protocol [7], [8] to send the new node, its parent, and its side (left or right child) to other replicas (line 29), which add the node to their own local trees (lines 30–38). We assume the causal broadcast protocol immediately delivers the message to the sender without waiting for any network communication.

A replica will not create a new node where it already has a same-side sibling, i.e., it will try to keep the tree binary. However, multiple replicas may concurrently insert nodes at the same position, creating same-side siblings like a and b in Fig. 3.

Delete: To implement $\text{delete}(i)$, a replica looks up the node at index i in the current list state, then causally broadcasts a message containing that element's ID (lines 40–41). All replicas then replace that node's value with a special value $\perp$ (line 44), flagging it as deleted (i.e., making it a *tombstone*). Nodes with this value are skipped when computing the externally visible list order and indexes (line 18); however, their non-deleted

Algorithm 1: Pseudocode for the Fugue Algorithm.

```

1: types:
2:   RID, type of replica identifiers
3:   ID := (RID × ℕ) ∪ {null}, type of element IDs
4:   V, type of values
5:   ⊥, a marker for deleted nodes
6:   {L, R}, type of a child node's side (left or right)
7:   NODE := ID × (V ∪ {⊥}) × ID × {L, R}, tree node tuples (id, value, parent, side)

8: per-replica CRDT state:
9:   replicaID ∈ RID: the unique ID of this replica
10:  tree ⊆ NODE × ID[] × ID[]: a set of tree node triples (node, leftChildren, rightChildren), initially {(root, [], [])}
    where root = (null, ⊥, null, null)
11:  counter ∈ ℕ: a counter for generating element IDs, initially 0

12: function values() : V[]
13:   return traverse(null)

14: function traverse(nodeID : ID) : V[]
15:   values ← []
16:   (node, leftChildren, rightChildren) ← the unique triple ∈ tree such that node.id = nodeID
17:   for childID ∈ leftChildren do values ← values + traverse(childID)
18:   if node.value ≠ ⊥ then values ← values + [node.value]
19:   for childID ∈ rightChildren do values ← values + traverse(childID)
20:   return values

21: function insert(i : ℕ, x : V)
22:   id ← (replicaID, counter); counter ← counter + 1
23:   leftOrigin ← node for (i − 1)-th value in values(), or root if i = 0
24:   rightOrigin ← next node after leftOrigin in the tree traversal that includes tombstones
25:   if ∄id', v'. (id', v', leftOrigin.id, R) ∈ tree then
26:     node ← (id, x, leftOrigin.id, R) // right child of leftOrigin; see Fig. 4(a)
27:   else
28:     node ← (id, x, rightOrigin.id, L) // left child of rightOrigin; see Fig. 4(b)
29:   broadcast (insert, node) by causal broadcast

30: on delivering (insert, node) by causal broadcast
31:   (parent, leftSibs, rightSibs) ← the unique triple ∈ tree such that parent.id = node.parent
32:   if node.side = R then
33:     i ← least index such that node.id < rightSibs[i]
34:     insert node.id into rightSibs at index i
35:   else
36:     i ← least index such that node.id < leftSibs[i]
37:     insert node.id into leftSibs at index i
38:   tree ← tree ∪ {(node, [], [])}

39: function delete(i : ℕ)
40:   node ← node for i-th value in values()
41:   broadcast (delete, node.id) by causal broadcast

42: on delivering (delete, id) by causal broadcast
43:   (node, _, _) ← the unique triple ∈ tree such that node.id = id
44:   node.value ← ⊥

```

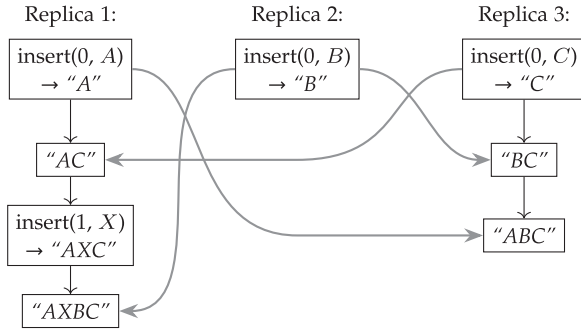


Fig. 6. An execution in which it is impossible to achieve both forward and backward non-interleaving, but maximal non-interleaving is possible.

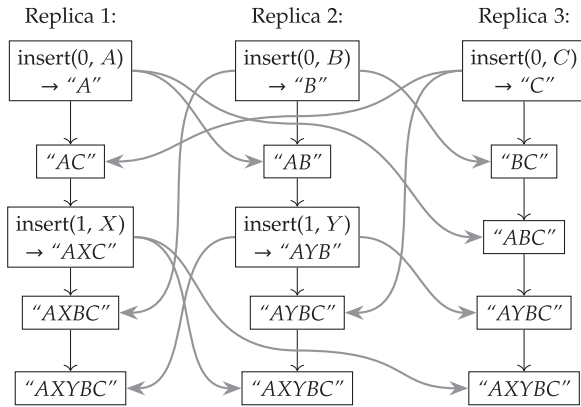


Fig. 7. An extension of the execution in Fig. 6, demonstrating the conditions of maximal non-interleaving.

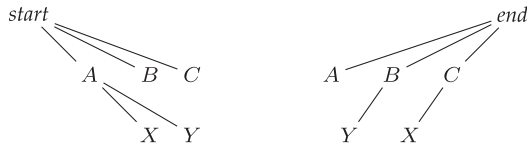


Fig. 8. Left- and right-origin trees for Fig. 7. The Fugue tree in this example is equivalent to the left-origin tree (all elements are right children of their parent).

causally later than B , we say that A and B are *concurrent*. This relation defines a partial order over list elements, and the causal broadcast protocol in Algorithm 1 delivers the insertion operations for those list elements in a linear extension of this partial order.

B. Defining Maximal Non-Interleaving

We already saw that the definition of non-interleaving by Kleppmann et al. [27] is impossible to satisfy (Section III-B). Finding a satisfiable definition requires some care. Let us first consider non-interleaving for forward insertions, like those in Fig. 1.

Definition 2: A replicated list algorithm is *forward non-interleaving* if it satisfies the strong list specification [4] and the following holds for all list elements A and B in all possible

list states: if A is the left origin of B , and B appears earlier in the list than any other element that has A as left origin, then A and B are consecutive list elements.

This definition captures the idea that elements A, B inserted in a forward sequence should be consecutive—no other elements may interleave with the sequence AB . In particular, this holds when B is the *only* element that has A as left origin. Once multiple elements B, C have left origin A , we cannot place all of them immediately after A . But we can still guarantee that sequences inserted after B and C respectively are not interleaved:

Lemma 3: Let a replicated list algorithm satisfy forward non-interleaving as defined in Definition 2. Then it is also satisfies Kleppmann et al.’s definition of forward non-interleaving [12, §4]: if two sequences $B_1 \dots B_m$ and $C_1 \dots C_n$ are inserted from left to right concurrently at the same starting position, then all B_j are on the same side of all C_i in the final list order.

Proof: Let $\prec$ be the global total order on list elements guaranteed by the strong list specification. Without loss of generality, $B_1 \prec C_1$. We need to show that $B_2, \dots, B_m \prec C_1$.

It is possible for a replica to be in a state that contains C_1 , B_2 , and all causally prior elements (including B_1), but no others. Since C_1 was inserted concurrently to B_1 , B_2 is the only element in this state that is causally later than B_1 . Thus it is the only element with left origin B_1 . Then by forward non-interleaving, B_1 and B_2 are consecutive. Together with $B_1 \prec C_1$, this implies $B_2 \prec C_1$. Because this relation holds for one possible replica in one state, the strong list specification requires it to hold in the final list order.

A similar argument shows that $B_3, \dots, B_m \prec C_1$.

It is tempting to define “backward non-interleaving” analogously to forward non-interleaving (replacing left origins with right origins), then define “non-interleaving” as the conjunction of forward and backward non-interleaving.

However, there are exceptional executions in which forward non-interleaving *forces* us to interleave backward insertions. Fig. 6 gives an example: Starting from an empty list, three replicas concurrently insert A , B , and C . Replica 3 receives all three elements and puts them in some order; without loss of generality, it is $A \prec B \prec C$. Replica 1 receives A and C , then inserts X in between those elements to obtain AXC . Finally, Replica 1 receives B .

Since X is the only element with left origin A , forward non-interleaving requires them to be consecutive: AX . Also, the strong list specification requires that since $A \prec B \prec C$ on Replica 3, all other replicas must place those elements in the same order. Thus the final list order must be $AXBC$. But then X and C are not consecutive, even though X is the only element with right-origin C . This rules out the above version of backward non-interleaving.

We do still want to mandate backward non-interleaving whenever it is possible. When forward and backward non-interleaving are in conflict with each other, we let forward non-interleaving take precedence, since forward (left-to-right) insertions are more common in text editors.

Definition 4: A replicated list algorithm is *maximally non-interleaving* if it satisfies the strong list specification [4] and the following holds for all list elements A and B in all possible list states:

- 1) (Forward non-interleaving) If A is the left origin of B , and B appears earlier in the list than any other element that has A as left origin, then A and B are consecutive list elements.
- 2) (Backward non-interleaving, with exceptions) If B is the right origin of A , and A appears later in the list than any other element that has B as right origin, then A and B are consecutive list elements, *unless* Lemma 5 below says otherwise.
- 3) If A and B have the same left origin and the same right origin, then the element with the lower ID appears earlier in the list.

Lemma 5: Given a replicated list algorithm that satisfies the strong list specification and forward non-interleaving (condition (1)). Suppose B is the right origin of A , and A appears later in the list than any other element that has B as right origin, *but*:

- i) A and B have different left origins; and
- ii) there is a C in the current list state such that $A.\text{leftOrigin} \prec C \prec B$ and C is not a descendant of $A.\text{leftOrigin}$ in the left-origin tree.

Then $A \prec C \prec B$, so A and B are not consecutive list elements.

We will prove Lemma 5 in the next section. For now, observe that the lemma forces $X \prec B \prec C$ in the example of Fig. 6. Thus the list order $AXBC$ is allowed by maximal non-interleaving: the fact that X and C are not consecutive is permitted due to the exception in condition (2).

Condition (3) is an arbitrary choice; two elements with the same left origin and the same right origin were inserted concurrently at the exact same place, so there is no reason to order them in a particular way. It turns out that this arbitrary choice is the *only* remaining degree of freedom after assuming (1) and (2): we show in Section V-E that maximal non-interleaving uniquely determines the list order.

Fig. 7 shows how maximal non-interleaving determines the list order in a more complex execution; see Fig. 8 for the final left- and right-origin trees. We start as in Fig. 6 with three replicas inserting A , B , and C concurrently into an empty list. These three elements have the same left and right origins (*start* and *end* respectively), so condition (3) requires them to be in ID order; without loss of generality, assume $A \prec B \prec C$. Replica 1 then receives $\{A, C\}$ and inserts X between them, while concurrently Replica 2 receives $\{A, B\}$ and inserts Y between them. We have already seen in Fig. 6 that in the state $AXBC$ on Replica 1, condition (1) requires AX to be consecutive, and thus $X \prec B$ in the list order. Moreover, Y is the only element with right origin B , so condition (2) requires YB to be consecutive (nothing contradicts this requirement). Finally, X is the only element with right origin C , but since condition Lemma 5 requires $X \prec B \prec C$, it is an exception from condition (2), and thus XC is not required to be consecutive. Thus, the only list order that satisfies maximal non-interleaving in this example is $AXYBC$.

C. From Fugue to FugueMax

It turns out that in executions like Fig. 7, Fugue might *not* satisfy maximal non-interleaving. Indeed, the previous paragraph explained that maximal non-interleaving implies $X \prec Y$. But in the Fugue tree (Fig. 8's left side), X and Y are same-side siblings, hence traversed in the lexicographic order of their IDs. This order might be $Y \prec X$.

We can repair this issue by changing the order of right-side siblings in the Fugue tree: when siblings X and Y have different right origins $C \neq B$, we sort X and Y by the *reverse* order of their right origins. For example, in Fig. 8's Fugue tree, $B \prec C$, so we should order $X \prec Y$. That allows Y to be consecutive with B , the leftmost right origin. We learned of this reverse-right-origin technique from Gentle's YjsMod [26].

Definition 6: *FugueMax* is the replicated list algorithm that is identical to Fugue except that its tree traversal visits right-side siblings in the *reverse* order of their right origins, breaking ties using the lexicographic order of their IDs.

Concretely, FugueMax's algorithm is identical to Fugue's Algorithm 1 except:

1. When generating a right child (Line 26), the node is additionally tagged with its right origin: $\text{node} \leftarrow (id, x, \text{leftOrigin.id}, R, \text{rightOrigin.id})$.
2. On delivering a right child (Line 33), it uses the sibling order described above: $i \leftarrow$ the least index such that $\text{node.rightOrigin} \succ \text{rightSibs}[i].\text{rightOrigin}$ **or** ($\text{node.rightOrigin} = \text{rightSibs}[i].\text{rightOrigin}$ **and** $\text{node.id} < \text{rightSibs}[i]$), where $\succ$ is the existing list order and $<$ is the lexicographic order on node IDs.

The proof that Fugue satisfies the strong list specification (Theorem 1) works identically for FugueMax.

We show in Section V-D that FugueMax is maximally non-interleaving and that Fugue is forward non-interleaving.

Moreover, one can check that in any situation where Fugue and FugueMax differ, some interleaving is inevitable, for any algorithm. Indeed, Fugue and FugueMax only differ in a situation analogous to Fig. 7: there must exist elements with the same left origin (X and Y) but different right origins (B and C). In such a situation, forward non-interleaving necessitates $\{X, Y\} < \{B, C\}$, but backward non-interleaving only allows the incompatible orders $YBXC$ or $XC YB$. (Consider tree traversals in Fig. 8.)

Such situations involve multiple interacting concurrent updates, which should be rare in practice. The advantage of FugueMax is only that it backward-interleaves one fewer pair of characters (here YB). Thus we believe that in practice, Fugue's simpler algorithm is worth its small amount of additional interleaving.

D. FugueMax is Maximally Non-Interleaving

The goal of this section is to prove that FugueMax is maximally non-interleaving (Theorem 9).

Recall that a depth-first pre-order traversal of a tree follows the rule: visit a node, then traverse its children in some order (whereby all descendants of a child are visited recursively before moving on to the next child).

Lemma 7: A replicated list algorithm that satisfies the strong list specification is forward non-interleaving if and only if its list order is some depth-first pre-order traversal of the left-origin tree.

Proof: ($\Leftarrow$): In any list state, let A and B be list elements such that A is the left origin of B , and B appears earlier in the list than any other element that has A as left origin. In the left-origin tree, B is the child of A that appears earliest in the list. Thus the list's depth-first pre-order traversal will visit B immediately after A : A and B are consecutive list elements.

($\Rightarrow$): To show that the list order is a depth-first pre-order traversal of the left-origin tree, it suffices to prove:

- a) If A is the parent of B in the left-origin tree, then $A \prec B$ in the list order.
- b) If A and D are siblings in the tree and $A \prec D$, then the entire subtree rooted at A precedes D in the list order.

For statement (a), A is the left origin of B , so $A \prec B$ by the strong list specification.

For statement (b), first observe that D is not causally later than A . We show this by contradiction: assume D is inserted in a list state that already contains A . Since A and D are siblings in the left-origin tree, $D.\text{leftOrigin} = A.\text{leftOrigin}$, and $A.\text{leftOrigin} \prec A$ in the list order. At the time D is inserted, $D.\text{leftOrigin}$ and D must be consecutive, implying $D \prec A$, which contradicts $A \prec D$.

Next, let B be a child of A in the left-origin tree. It is possible for a replica to be in a state that contains B , D , and all causally prior elements (including A), but no others. Every element $E \notin \{B, D\}$ in this state is causally prior to either B or D ; however, if E is causally later than A , then E cannot be causally prior to D , since this would contradict the previous paragraph's finding that A is not causally prior to D . Therefore, any element of this state that is causally later than A and not equal to B must be causally prior to B . Any element whose left origin is A must be causally later than A . Hence B appears earlier in the list than any other element whose left origin is A . By forward non-interleaving, A and B are consecutive in this state. Thus $B \prec D$.

One can similarly prove that $B \prec D$ whenever B is a grandchild, great-grandchild, etc. of A , using induction on the depth of B . Hence all of the elements in the subtree rooted at A appear before D in the list order. $\square$

Proof of Lemma 5: By Lemma 7, the list order must be some depth-first pre-order traversal of the left-origin tree. In any such traversal, C appears later in the list order than the entire subtree rooted at $A.\text{leftOrigin}$, because it appears later than $A.\text{leftOrigin}$ and is not part of its subtree. In particular, $A \prec C$. Meanwhile, $C \prec B$ by assumption. $\square$

Lemma 8: In any state of FugueMax (resp., Fugue), we have the following facts about left origins.

- a) The left origin of an element D is given by: starting at D , walk up the FugueMax tree until you encounter a node that is a right child of its parent; that node's parent is D 's left origin.
- b) Let A and B be list elements such that $A \prec B$. Then B is a descendant of A in the FugueMax tree if and only if B is a descendant of A in the left-origin tree.

Proof. a): Let E be the alleged left origin for D . In the state just after inserting D , list elements E and D are consecutive in FugueMax's tree traversal. Since FugueMax satisfies the strong list specification (by the same proof as Theorem 1), E must be D 's left origin.

b): ($\Leftarrow$): Assume B is a child of A in the left-origin tree, that is, B was immediately after A in the list order when B was inserted. Then the FugueMax algorithm either made B a right child of A in the FugueMax tree (if A had no right child at the time of inserting B), or a left child of a right descendant of A (if A had at least one right child). In either case, B is a descendant of A in the FugueMax tree. By induction we generalize to the case when B is a descendant of A in the left-origin tree, using a similar argument at each tree level.

($\Rightarrow$): Assume B is a descendant of A in the FugueMax tree and $A \prec B$. If B is a left child of its parent, B appears before its parent in the list order, so the parent cannot be A . Walking up the tree from B , since there is an element before B in the list order, we must eventually reach an element that is a right child of its parent P . By (a), P must be the left origin of B . Either $P = A$, in which case B is a child of A in the left-origin tree, or (by induction) P is a descendant of A in the left-origin tree.

Theorem 9: FugueMax satisfies conditions (1), (2), and (3) of Definition 4. Hence FugueMax is maximally non-interleaving.

Proof. Condition (1): Let A and B have the form in condition (1): A is the left origin of B , and B appears earlier in the list than any other element that has A as left origin. We must prove that A and B are consecutive.

By Lemma 8(a), in the FugueMax tree, B is the earliest among all nodes that are either a right child of A or a left descendant of a right child of A . This is precisely the first node that FugueMax's tree traversal visits when traversing the right children of A . Thus the traversal visits B immediately after A : they are consecutive.

Condition (2): Let A and B have the form in condition (2): B is the right origin of A , A appears later in the list than any other element that has B as right origin, and conditions (i) and (ii) in Lemma 5 are not both true. We must prove that A and B are consecutive.

First suppose (i) is false, so that A and B have the same left origins. By Lemma 8(b), B is a descendant of $A.\text{leftOrigin}$ in the FugueMax tree, so $A.\text{leftOrigin}$ must have at least one right child in the FugueMax tree. Thus, when A was inserted between $A.\text{leftOrigin}$ and B , FugueMax's insert function made A a left child of B .

Because A is the last element in the list order with right origin B , A is also the last to be traversed of the left children of B in the FugueMax tree. Hence if A has no right children of its own, then FugueMax's tree traversal visits B immediately after A : they are consecutive. Otherwise, let D be a right child of A such that A did not have any other right child when D was inserted. Immediately before the insertion of D , A did not have a right-side child, so A and B were consecutive. Hence D 's right origin is B and $A \prec D$, contrary to assumption.

Next, suppose (i) is true and A and B are not consecutive. We must prove that there exists a C of the form given in (ii). Since A and B are not consecutive, there exist one or more elements

in between A and B ; take C to be an in-between element that is not causally later than any other element between A and B . We have $A.\text{leftOrigin} \prec C \prec B$ because $A.\text{leftOrigin} \prec A$. It remains to prove that C is *not* a descendant of $A.\text{leftOrigin}$ in the left-origin tree.

We use proof by contradiction: assume that C is a descendant of $A.\text{leftOrigin}$ in the left-origin tree. Thus C is either a descendant of A or a sibling (it cannot be a descendant of a sibling, since otherwise the sibling would be a causally prior element between A and B).

Case C is a descendant of A in the left-origin tree. In this case, C is causally later than both A and B and was inserted in between them. Because A appears later in the list than any other element that has B as right origin, C 's right origin is not B ; instead, it must be some other element between A and B . But then C 's right origin is an in-between element that is causally prior to C , contradicting our choice of C .

Case C is a sibling of A in the left-origin tree. That is, A and C have the same left origins.

At the time of A 's insertion, $A.\text{leftOrigin}$ and B were consecutive, and A was inserted between them. By (i), B 's left origin is not $A.\text{leftOrigin}$; thus, B is not a descendant of $A.\text{leftOrigin}$ in the left-origin tree; and by Lemma 8(b), B is not a descendant of $A.\text{leftOrigin}$ in the FugueMax tree either. Therefore $A.\text{leftOrigin}$ did not have any right children in the FugueMax tree at the time when A was inserted, and FugueMax's insert function made A a right child of $A.\text{leftOrigin}$.

C is also a right child of $A.\text{leftOrigin}$ in the FugueMax tree: it is a descendant by Lemma 8(b); and if it were not a child, then its FugueMax parent would be a causally prior in-between element. Also, by assumption, C 's right origin is not B .

Thus A and C are right-side siblings in the FugueMax tree with different right origins. This situation is similar to elements X and Y in Fig. 8. FugueMax's tree traversal then visits them in the reverse order of their right origins. Since $A \prec C$, this implies that C 's right origin appears before B in the list order. But then C 's right origin is a causally prior in-between element, again contradicting our choice of C .

Condition (3): Let A and B have the same left origin and the same right origin. Then FugueMax's insert function assigned them the same parent and side. FugueMax traverses these same-side siblings, with equal right origins, in order by ID. $\square$

E. Uniqueness of Maximally Non-Interleaving Order

We finish by showing that maximal non-interleaving uniquely determines the list order. Specifically, any maximally non-interleaving replicated list induces the same total order on elements as FugueMax. This reinforces our claim that FugueMax is "maximally" non-interleaving—any additional nontrivial constraints would be impossible to satisfy.

Theorem 10: Let L be a replicated list algorithm that is maximally non-interleaving. Then L is semantically equivalent to FugueMax. That is, in any execution, L orders its list of elements according to FugueMax's global total order $\prec$ on elements.

Proof: We will show that conditions (1), (2), and (3) of Definition 4 force L to use a specific list order. Since FugueMax

is maximally non-interleaving, this list order must be the same as FugueMax's.

First, condition (1) and Lemma 7 imply that L 's list order is some depth-first pre-order traversal of the left-origin tree. Thus the only remaining degree of freedom for L 's list order is the order of siblings within the left-origin tree.

So, let P be any element of L or $start$, and let S_P be the set of list elements whose left origin is P . Consider the forest of S_P elements in which each element's parent is its right origin, except that an element has no parent if its right origin is not in S_P , or if its right origin is end .

For any parent-child edge (A, B) in S_P , A and B have the same left origin (P). Thus A and B are *not* an exception to condition (2). A mirrored version of the proof of Lemma 7 shows that L 's list order, when restricted to any tree T in S_P 's forest, must be a depth-first post-order traversal of T . (A depth-first post-order traversal satisfies the rule: traverse a node's children in some order, then visit the node.) Siblings within T must be ordered by their IDs: they have the same left origins (P) and right origins (their parent), so condition (3) applies. Conditions (2) and (3) thus fully determine the order of elements in the same right-origin tree T .

It remains to specify the relative order of S_P elements that are in different trees. We claim:

- a) If D and E are tree roots in S_P with $D \prec E$, then the entire tree rooted at D (in S_P) appears before the entire tree rooted at E in the list order.
- b) L orders the tree roots in S_P identically to how FugueMax sorts a node's right-side children: in the reverse order of their right origins, breaking ties by ID. (One can show that the tree roots in S_P are precisely the right-side children of P in FugueMax's tree.)

To prove these claims, we first argue that all tree roots in S_P were inserted concurrently. Indeed, in any state that already contains a tree root (more generally, an element with left origin P), the element R immediately following P must also have left origin P , because L 's list order is a depth-first pre-order traversal of the left-origin tree. A new element inserted into this state with left origin P will have right origin $R \in S_P$, so it is not a tree root in S_P .

We prove the two claims above in turn:

(a) Let A and D be tree roots in S_P such that $D \prec A$ in L , and let B be a child of A . By the previous paragraph, D is not causally later than A . Then the claim follows by a mirrored version of the last two paragraphs in Lemma 7's proof (using condition (2) in place of forward non-interleaving).

(b) Let D and E be distinct tree roots in S_P . If D and E have the same right origins, then by condition (3), L must order them by ID. Otherwise, let R_D and R_E be their right origins, and assume $R_E \prec R_D$. We need to show that $D \prec E$.

It is possible for a replica to be in a state σ that contains D , E , and all causally prior elements, but no others. Since E was inserted directly between P and R_E , no elements causally prior to E are between P and R_E . Likewise, no elements causally prior to D are between P and R_D , hence none are between P and $R_E \prec R_D$. Thus, D and E are the only elements in state σ that can be between P and R_E .

D and E are indeed between P and R_E : because all tree roots in S_P were inserted concurrently, D and E are the only elements in σ with left origin P ; thus they immediately follow P in L 's depth-first pre-order traversal of the left-origin tree. So, state σ contains either $PDER_E$ or $PEDR_E$ as a contiguous subsequence. Condition (2) forces L to use $PDER_E$: E is the last element in the list order with R_E as its right origin, and there is no C that could grant us an exception, so E and R_E must be consecutive. (By the same argument, YB must be consecutive in Fig. 7.) Hence $D \prec E$. $\square$

Observe that the above proof gives an alternate characterization of FugueMax's list order:

- 1) First, order elements by a depth-first pre-order traversal of the left-origin tree.
- 2) Second, order siblings within that tree by a depth-first post-order traversal of their right-origin forest.
- 3) Third, order roots within this forest that have different right origins by the reverse order of their right origins. (In Fugue, instead order them by ID.)
- 4) Fourth, order roots within this forest that have the same right origin, and all other siblings in the forest, by ID.

We chose to present FugueMax and Fugue as a double-sided tree for simplicity. But one could just as well use an algorithm based on the above characterization.

VI. IMPLEMENTATION AND EVALUATION

We implemented two variations of Fugue, and one of FugueMax, in TypeScript. Each is written as a custom CRDT for the Collabs library [31]; Collabs provides causal order delivery and other utilities. The variations are:

- *Fugue*: An optimized implementation of Algorithm 1 in 1132 lines of code. It uses practical optimizations inspired by Yjs [32] and RGASplit [33]. In particular, it condenses sequentially-inserted tree nodes into a single “waypoint” object instead of one object per node, and it uses Protocol Buffers to efficiently encode update messages and saved documents. Collabs v0.6.1 uses this implementation for its list CRDTs.
- *Fugue Simple*: A direct implementation of Algorithm 1 in 298 lines of code. It represents the state as a doubly-linked tree with one object per node, and it uses GZIP'd JSON encodings.
- *FugueMax Simple*: A direct implementation of Algorithm 1 with FugueMax's modifications (see Section V-C) in 435 lines of code.

A. Benchmarks

We evaluated our implementations using a modified version of Jahns's crdt-benchmarks [34]. Our Fugue and FugueMax implementations, benchmark code, and raw data are available on GitHub.¹

Our goal with these benchmarks is to show that Fugue and FugueMax can be implemented with practical performance, comparable to existing CRDT implementations, in spite of the constraints imposed by non-interleaving. All experiments used

TABLE II
SAVED DOCUMENT METRICS. THE PLAIN TEXT (WITHOUT CRDT METADATA OR TOMBSTONES) IS 105 KB IN SIZE

Implementation	Save size	Save time	Load time
Fugue	168 ± 0 kB	20 ± 1 ms	13 ± 2 ms
Fugue Simple	1,021 ± 0 kB	583 ± 5 ms	334 ± 3 ms
FugueMax Simple	1,237 ± 0 kB	788 ± 11 ms	522 ± 7 ms
Automerge-Wasm	129 ± 0 kB	180 ± 0 ms	2,746 ± 6 ms
Yjs	160 ± 0 kB	17 ± 1 ms	63 ± 6 ms
Y-Wasm	160 ± 0 kB	5 ± 1 ms	15 ± 0 ms

Node.js v18.15.0 running on an Ubuntu 22.04.3 LTS desktop with a 4-core Intel i7 CPU @1.90 GHz and 16 GB RAM. For each metric, we performed 5 warmup trials followed by 10 measured trials; tables show mean ± standard deviation for the 10 measured trials.

We also compared to existing implementations in the crdt-benchmarks repository:

- **Automerge-Wasm** (v0.5.0) is a CRDT with a JSON data model, implemented in Rust and compiled to WebAssembly.² Its list CRDT is based on RGA [20].
- **Yjs** (v13.6.8) is a JavaScript CRDT library [24]. Its list datatype is based on YATA [35], and it is known for its good performance [32].
- **Y-Wasm** (v0.16.10) is a Rust-to-WebAssembly variant of Yjs.³

Tables II and III show results from a benchmark that replays a real-world text-editing trace [36], in which every keystroke of the writing process for the LaTeX source of a 17-page paper [37] was captured. It consists of 182,315 single-character insert operations and 77,463 single-character delete operations, resulting in a final document size of 104,852 characters (not including tombstones). Each implementation processed the full trace sequentially on a single replica. Results for additional benchmarks, including microbenchmarks with concurrent operations, can be found in our GitHub repository.

Table II considers the final saved document including CRDT metadata. In a typical collaborative app, this saved document would be saved (possibly on a server) at the end of each user session, and loaded at the start of the next session. Thus save size determines disk/network usage, while save/load time determines user-perceived save and startup latencies.

We see that Fugue is comparable to state-of-the-art Yjs on all three metrics, and the CRDT metadata is only 60% of the literal text's size. Fugue Simple and FugueMax Simple are worse but still usable in practice. FugueMax Simple has a larger save size than Fugue Simple because it must additionally track right-side children's right origins.

Table III shows performance metrics for live usage by a single user. Memory usage shows the increase in heap used⁴ from the start to the end of the trace and thus approximates each list

²<https://github.com/automerge/automerge>

³<https://github.com/y-crdt/y-crdt>

⁴Measured with Node.js's `process.memoryUsage().heapUsed` after garbage collection. We exclude WebAssembly libraries because they do not use the JavaScript heap. (While resident set size measures WebAssembly memory usage, it had frequent outliers and gave implausible values for Automerge-Wasm.)

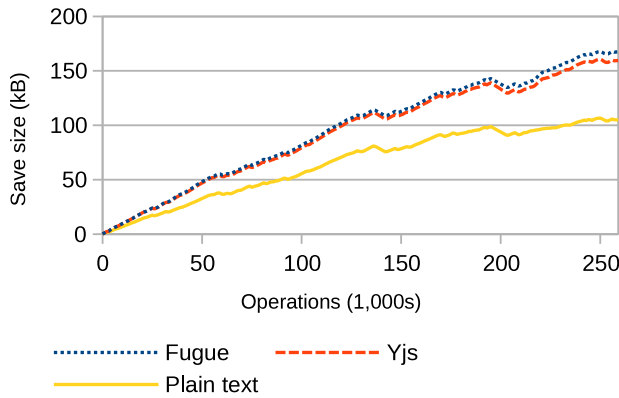
¹<https://github.com/mweidner037/fugue>

TABLE III
METRICS FOR REPLAYING A CHARACTER-BY-CHARACTER TEXT EDITING TRACE WITH 260 K OPERATIONS

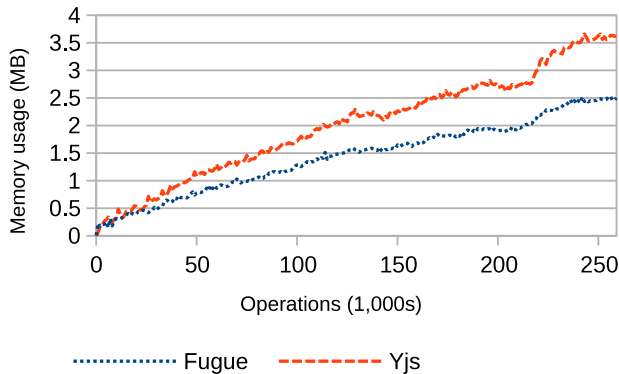
Implementation	Memory usage (MB)	Network bytes/op	Ops/sec (1,000s)
Fugue	2.4 ± 0.0	46 ± 0	94 ± 5
Fugue Simple	64.8 ± 0.0	151 ± 0	17 ± 0
FugueMax Simple	71.9 ± 0.0	188 ± 0	16 ± 0
Automerge-Wasm	—	126 ± 0	52 ± 0
Yjs	3.3 ± 0.2	29 ± 0	39 ± 0
Y-Wasm	—	29 ± 0	7 ± 0

TABLE IV
METRICS FOR THE REAL TEXT TRACE REPEATED 100 TIMES SEQUENTIALLY. THE LITERAL TEXT HAS SIZE 10,485 Kb. WE EXCLUDE IMPLEMENTATIONS THAT USE EXCESSIVE TIME OR MEMORY

Implementation	Save size (kB)	Save time (ms)	Load time (ms)	Memory usage (MB)
Fugue	$17,845 \pm 0$	$1,405 \pm 35$	640 ± 8	223 ± 0
Yjs	$15,989 \pm 0$	479 ± 35	$2,316 \pm 423$	294 ± 17



(a) Saved document size.



(b) Memory usage.

Fig. 9. Metrics as a function of progress through the real text trace.

CRDT's in-memory size. Network bytes/op shows the average size of the per-op messages sent to remote collaborators. Ops/sec shows the average operation throughput; it reflects the time to process an op and encode a message for remote collaborators. For example, Fugue achieves 94,000 ops/sec, an average of 11 μ s per operation.

We again see that Fugue is practical and comparable to Yjs. In particular, its memory usage is only a few MB—about 23 bytes per character, or 13 bytes per characters-including-tombstones.

This refutes a common criticism of CRDTs for collaborative text editing: namely, that they have too much per-character memory overhead [38]. The memory overhead is worse for Fugue Simple (618 bytes/char) and FugueMax Simple (685 bytes/char), but the total is still well within modern memory limits. For all Fugue variants, the network usage and operation throughput are far from being bottlenecks, given that a typical user types at ≈ 10 chars/sec and a typical collaborative document has < 100 simultaneous users.

Fig. 9(a) and (b) show how save size and memory usage vary throughout the text editing trace. The size of the plain text (without CRDT metadata or tombstones) is given for comparison. Observe that both metrics track the plain text's size at a modest multiple, and save size even decreases when text is deleted, despite tombstones.

Finally, Table IV shows selected metrics for the same text-editing trace but repeated 100 times. The final document contains 10.5 million characters—several times longer than Tolstoy's *War and Peace*. Nonetheless, Fugue's performance remains tolerable: 18 MB save size, less than 2 seconds to save or load, and 223 MB memory usage. Additionally, average network usage and throughput (not shown) remain within a $2\times$ factor of Table III.

VII. CONCLUSION

Interleaving of concurrent insertions at the same position is an undesirable but largely ignored problem with many replicated list algorithms that are used for collaborative text editing. Indeed, most CRDT and OT algorithms that we surveyed exhibit interleaving anomalies. We also found that existing definitions of non-interleaving are impossible to satisfy.

In this paper, we proposed a new definition, maximal non-interleaving. We also introduced the Fugue and FugueMax list CRDTs and proved that FugueMax satisfies maximal non-interleaving, while Fugue is simpler and falls only slightly short of it. Our optimized implementation of Fugue has performance comparable to the state-of-the-art Yjs library.

In future work we plan to formally analyze Sync9 [25] and Gentle's modified version of the Yjs algorithm (YjsMod) [26]. We conjecture that Sync9 is semantically equivalent to Fugue,

while YjsMod is semantically equivalent to FugueMax. If so, then YjsMod is also maximally non-interleaving. These algorithms are also CRDTs; in future work it would be interesting to investigate whether maximally non-interleaving Operational Transformation algorithms exist.

ACKNOWLEDGMENT

We thank Seph Gentle and Aryan Shah for insightful discussions and feedback on a draft of this paper.

REFERENCES

- [1] C. A. Ellis and S. J. Gibbs, "Concurrency control in groupware systems," in *Proc. ACM Int. Conf. Manage. Data*, 1989, pp. 399–407.
- [2] D. A. Nichols, P. Curtis, M. Dixon, and J. Lamping, "High-latency, low-bandwidth windowing in the Jupiter collaboration system," in *Proc. 8th Annu. ACM Symp. User Interface Softw. Technol.*, 1995, pp. 111–120.
- [3] C. Sun, X. Jia, Y. Zhang, Y. Yang, and D. Chen, "Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems," *ACM Trans. Comput.-Hum. Interaction*, vol. 5, no. 1, pp. 63–108, Mar. 1998.
- [4] H. Attiya, S. Burckhardt, A. Gotsman, A. Morrison, H. Yang, and M. Zawirski, "Specification and complexity of collaborative text editing," in *Proc. 2016 ACM Symp. Princ. Distrib. Comput.*, 2016, pp. 259–268.
- [5] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in *Proc. 13th Int. Symp. Stabilization Saf. Secur. Distrib. Syst.*, 2011, pp. 386–400.
- [6] N. Preguiça, C. Baquero, and M. Shapiro, *Encyclopedia of Big Data Technologies*. Berlin, Germany: Springer, 2018.
- [7] K. P. Birman, A. Schiper, and P. Stephenson, "Lightweight causal and atomic group multicast," *ACM Trans. Comput. Syst.*, vol. 9, no. 3, pp. 272–314, Aug. 1991.
- [8] C. Cachin, R. Guerraoui, and L. Rodrigues, *Introduction to Reliable and Secure Distributed Programming*, 2nd ed. Berlin, Germany: Springer, 2011.
- [9] N. Preguiça, J. M. Marques, M. Shapiro, and M. Letia, "A commutative replicated data type for cooperative editing," in *Proc. 29th IEEE Int. Conf. Distrib. Comput. Syst.*, 2009, pp. 395–403.
- [10] S. Weiss, P. Urso, and P. Molli, "Logoot: A scalable optimistic replication algorithm for collaborative editing on P2P networks," in *Proc. 29th IEEE Int. Conf. Distrib. Comput. Syst.*, 2009, pp. 404–412.
- [11] B. Nédelec, P. Molli, A. Mostefaoui, and E. Desmontils, "LSEQ: An adaptive structure for sequences in distributed collaborative editing," in *Proc. ACM Symp. Document Eng.*, 2013, pp. 37–46.
- [12] M. Kleppmann, V. B. F. Gomes, D. P. Mulligan, and A. R. Beresford, "OpSets: Sequential specifications for replicated datatypes (extended version)," 2018, *arXiv:1805.04263*.
- [13] C. Sun, D. Sun, Agustina, and W. Cai, "Real differences between OT and CRDT for co-editors," Oct. 2018, *arXiv:1810.02137*.
- [14] M. Ressel, D. Nitsche-Ruhland, and R. Gunzenhäuser, "An integrating, transformation-oriented approach to concurrency control and undo in group editors," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 1996, pp. 288–297.
- [15] C. Sun and C. Ellis, "Operational transformation in real-time group editors: Issues, algorithms, and achievements," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 1998, pp. 59–68.
- [16] J. Day-Richter, "What's different about the new Google docs: Making collaboration fast," Sep. 2010. [Online]. Available: <https://drive.googleblog.com/2010/09/whats-different-about-new-google-docs.html>
- [17] A. Imine, P. Molli, G. Oster, and M. Rusinowitch, "Proving correctness of transformation functions in real-time groupware," in *Proc. 8th Eur. Conf. Comput. Supported Cooperative Work*, 2003, pp. 277–293.
- [18] G. Oster, P. Urso, P. Molli, and A. Imine, "Proving correctness of transformation functions in collaborative editing systems," INRIA, Le Chesnay-Rocquencourt, France, Tech. Rep. RR-5795, 2005. [Online]. Available: <https://hal.inria.fr/inria-00071213/>
- [19] G. Oster, P. Urso, P. Molli, and A. Imine, "Data consistency for P2P collaborative editing," in *Proc. ACM Conf. Comput. Supported Cooperative Work*, 2006, pp. 259–268.
- [20] H.-G. Roh, M. Jeon, J.-S. Kim, and J. Lee, "Replicated abstract data types: Building blocks for collaborative applications," *J. Parallel Distrib. Comput.*, vol. 71, no. 3, pp. 354–368, 2011.
- [21] M. Kleppmann, "Insertion interleaving test," Feb. 2018. [Online]. Available: <https://github.com/ept/insert-interleaving>
- [22] M. Suleiman, M. Cart, and J. Ferrié, "Serialization of concurrent operations in a distributed collaborative environment," in *Proc. ACM Int. Conf. Supporting Group Work*, Nov. 1997, pp. 435–445.
- [23] G. Oster, P. Molli, P. Urso, and A. Imine, "Tombstone transformation functions for ensuring consistency in collaborative editing systems," in *Proc. 9th IEEE Int. Conf. Collaborative Comput.*, 2006, pp. 1–10.
- [24] K. Jahns, "Yjs," GitHub repository, Dec. 2022. [Online]. Available: <https://github.com/yjs/yjs>
- [25] G. Little and M. Toomim, "Sync9: A peer-to-peer synchronizer," 2021. [Online]. Available: <https://github.com/braid-org/braidjs/blob/21df3b0/sync9/sync9.js>
- [26] S. Gentle, "Reference CRDTs," GitHub repository, Oct. 2021. [Online]. Available: <https://github.com/josephg/reference-crdts>
- [27] M. Kleppmann, V. B. F. Gomes, D. P. Mulligan, and A. R. Beresford, "Interleaving anomalies in collaborative text editors," in *Proc. 6th Workshop Princ. Pract. Consistency Distrib. Data*, Mar. 2019, pp. 1–7.
- [28] A. R. Chandrassery, "Personal communication," 2021.
- [29] M. Kleppmann, "Moving elements in list CRDTs," in *Proc. 7th Workshop Princ. Pract. Consistency Distrib. Data*, 2020, pp. 1–6.
- [30] V. Grishchenko, "Citrea and swarm: Partially ordered op logs in the browser: Implementing a collaborative editor and an object sync library in JavaScript," in *Proc. 1st Workshop Princ. Pract. Eventual Consistency*, 2014, pp. 1–3.
- [31] M. Weidner et al., "Collabs," GitHub repository, Sep. 2023. [Online]. Available: <https://github.com/composablesys/collabs>
- [32] K. Jahns, "Are CRDTs suitable for shared editing?" Blog post, Aug. 2020. [Online]. Available: <https://blog.kevinjahns.de/are-crdts-suitable-for-shared-editing/>
- [33] L. Briot, P. Urso, and M. Shapiro, "High responsiveness for group editing CRDTs," in *Proc. 19th Int. Conf. Supporting Group Work*, Nov. 2016, pp. 51–60.
- [34] K. Jahns, "CRDT benchmarks," 2022. [Online]. Available: <https://github.com/dmonad/crdt-benchmarks/>
- [35] P. Nicolaescu, K. Jahns, M. Derntl, and R. Klamma, "Near real-time peer-to-peer shared editing on extensible data types," in *Proc. 19th Int. Conf. Supporting Group Work*, Nov. 2016, pp. 39–49.
- [36] M. Kleppmann, "Benchmarking resources for automeerge," 2020. [Online]. Available: <https://github.com/automeerge/automeerge-perf>
- [37] M. Kleppmann and A. R. Beresford, "A conflict-free replicated JSON datatype," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 10, pp. 2733–2746, Oct. 2017.
- [38] D. Sun, C. Sun, A. Ng, and W. Cai, "Real differences between OT and CRDT in correctness and complexity for consistency maintenance in co-editors," *Proc. ACM Hum.-Comput. Interaction*, vol. 4, May 2020, Art. no. 21.



Matthew Weidner received the MPhil degree from the University of Cambridge, Cambridge, U.K., investigating decentralized cryptographic protocols. He is currently working toward the PhD degree with Carnegie Mellon University, Pittsburgh, PA, USA, advised by Heather Miller. His research interests include distributed systems, data structures, and API design, with a focus on making CRDTs more flexible and easier to use.



Martin Kleppmann is currently an associate professor with the University of Cambridge, Cambridge, U.K. He was a software engineer and entrepreneur with Internet companies including Rapportive and LinkedIn. His research interests include decentralisation and distributed systems security, in particular local-first collaboration software, CRDTs, and formal verification. His book *Designing Data-Intensive Applications* (O'Reilly Media) is the most popular text on data systems architecture, and it has been translated into eight languages.